

Chapter 18: GPU Serving Mechanics: PagedAttention, Continuous Batching, and vLLM

Preceding Bridge: In [Chapter 12: Production Agent Deployment](#), you learned how to stream LLM responses and manage user sessions. But how does an inference engine like vLLM or TensorRT-LLM actually execute transformer models on physical Nvidia GPUs? In this chapter, we explore the hardware-level bottleneck of modern AI: the **KV Cache**, **PagedAttention**, **Continuous Batching**, and **Prefix Caching**.

1. Plain-English Jargon Demystifier

Technical Term	Plain English Translation	Real-World Metaphor
Prefill Phase	The GPU reading and encoding the entire input prompt all at once.	Reading an entire legal contract before beginning to speak. Compute-heavy!
Decode Phase	The GPU generating one single output token at a time, autoregressively.	Speaking one word every second. Memory bandwidth-bound!
KV Cache	Storing Key and Value vectors for past tokens in GPU VRAM so we don't recalculate them on every new token.	A chalkboard where you keep notes on what you've already said so you don't repeat yourself.
Memory Fragmentation	Wasted GPU memory caused by reserving large blocks that aren't fully filled.	Reserving an entire 8-person dining table for a single diner who might invite friends later.
PagedAttention	Dividing the KV cache into small fixed-size virtual memory pages (e.g., 16 tokens/block), eliminating fragmentation.	Booking dining hall seats one chair at a time as guests actually arrive at the door.
Continuous Batching	Evicting completed requests immediately and inserting new requests at each token generation step.	A fast-moving conveyor belt where finished packages leave and new packages join instantly.
Prefix Caching	Reusing the pre-computed KV Cache of a shared system prompt across hundreds of users.	Printing 1,000 copies of the syllabus once, instead of writing it out by hand for every student.

2. Spoon-Fed Mental Model: The Restaurant Reservation Problem

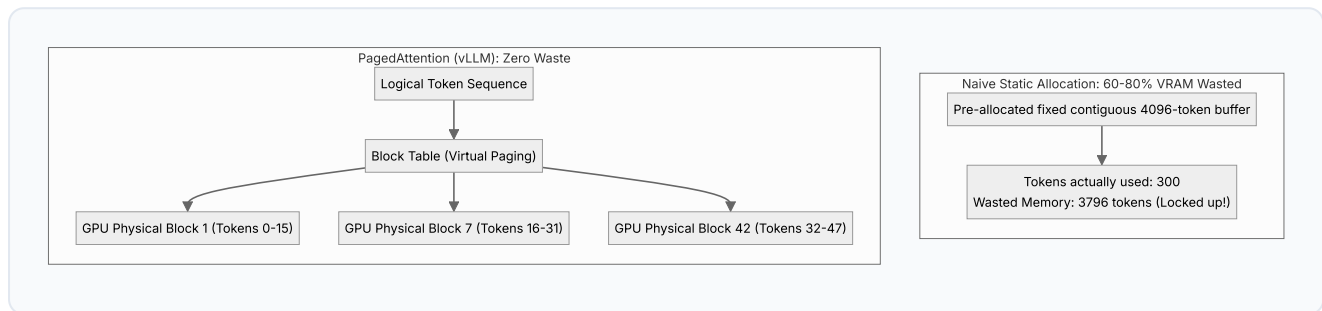
Imagine an expensive restaurant with 80 seats.

The Naive Serving Model (Static Batching):

- When 4 guests walk in, the host asks: *"What is the maximum number of dishes you could possibly order?"*
- Guests reply: *"Maybe 50 dishes, maybe only 2."*
- The host locks down 50 tables just in case, and **refuses to seat any other customers until those 4 guests pay and leave!**
- Result: **75% of tables sit completely empty and wasted**, and outside lines grow out of control.

The PagedAttention Model (vLLM):

- The host gives each customer a table for just **1 dish at a time**.
- As the customer finishes a dish and orders another, the waiter pulls up an extra modular table segment from storage.
- When a customer finishes their meal, their tables are immediately reassigned to waiting patrons in the lobby!
- Result: **Near 100% table utilization**, zero wasted space, and 5x higher throughput!



3. The Mathematics of KV Cache Memory Consumption

Why does GPU memory (VRAM) run out so quickly during inference? For every token processed, the transformer computes Key and Value vectors for all attention heads across all layers.

The formula for KV Cache size is:

$$\text{KV Cache Bytes} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times \text{seq_len} \times \text{bytes_per_element}$$

Real-World Example: Llama-3-70B (FP16, 2 bytes/float)

- Layers (n_{layers}): 80
- KV Heads (n_{heads}): 8 (Grouped-Query Attention)
- Head Dimension (d_{head}): 128
- Bytes per element: 2 (FP16)

$$\text{Per Token Memory} = 2 \times 80 \times 8 \times 128 \times 2 = 327,680 \text{ bytes} \approx 320 \text{ KB per token}$$

For a single user generating a **4,096-token sequence**:

$$\text{VRAM required per user} = 4,096 \times 320 \text{ KB} \approx 1.28 \text{ GB}$$

If you want to serve **100 concurrent users**, you need **128 GB of VRAM solely for the KV Cache**, on top of the 140 GB needed just to store the model weights!

4. Continuous Batching vs Static Batching

In traditional DL frameworks (HuggingFace transformers), batching is static:

- Request A finishes in 50 tokens.
- Request B takes 1,000 tokens.
- Request A's GPU thread must sit completely idle for 950 steps waiting for Request B to finish!

Continuous (Iteration-Level) Batching schedules at the granularity of a **single token iteration**:

1. At step T , Request A generates its `<endof text>` token and is instantly evicted.
 2. Request C from the waiting queue is slotted into Request A's vacant slot immediately.
 3. GPU cores operate at 95%+ compute saturation continuously!
-

5. Junior vs Staff Implementation: LLM Serving



6. Complete Runnable Implementation: KV Cache Paging Simulator

Here is a pure Python simulation demonstrating how PagedAttention allocates non-contiguous memory blocks and tracks virtual-to-physical block tables:



7. Chapter Milestone Check

Verify your understanding before moving forward:

1. **Why does the autoregressive Decode phase of an LLM spend most of its time waiting on GPU memory bandwidth rather than compute?**
 - *Answer:* For each new single token, the GPU must stream billions of model weights and millions of past KV cache values from high-bandwidth memory (HBM) into SRAM registers just to perform a handful of matrix-vector multiplications.
2. **How does PagedAttention eliminate the 60-80% memory waste common in naive static serving?**

- *Answer:* Instead of pre-allocating a contiguous array for the worst-case sequence length (e.g., 4096 tokens), PagedAttention allocates small virtual blocks (e.g., 16 tokens) dynamically as tokens are generated, mapping non-contiguous physical blocks via a block table.

3. What is Prefix Caching and why is it game-changing for agentic multi-turn chats?

- *Answer:* Multi-turn agents repeatedly send the same system prompt and tools list. Prefix caching retains the KV cache of that shared prefix across requests, dropping the time-to-first-token (TTFT) and compute cost to near zero.

Further Reading

- [PagedAttention paper \(vLLM\)](#)
 - [vLLM documentation](#)
 - [NVIDIA: mastering LLM inference optimisation](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. What problem does PagedAttention solve?

KV-cache memory fragmentation: it stores the cache in fixed-size blocks allocated on demand, raising batch size and throughput.

▼ 2. Prefill versus decode?

Prefill processes the prompt in parallel (compute-bound); decode generates tokens one at a time (memory-bound).

▼ 3. What does continuous batching do?

Re-forms the batch every decoding step so finished sequences leave and new ones join immediately.

▼ 4. How do you estimate KV cache size per token?

`2 x layers x kv_heads x head_dim x bytes` (e.g. 128 KiB per token for Llama-3 8B in FP16).